LoRA + DoRA as Extension: Weight-Decomposed Low-Rank Adaptation on GPT-2

Arnav Tevatia (at846) | Cornell University | CS 4782 Deep Learning | Spring 2026

Papers: Hu et al. LoRA, ICLR 2022 | Liu et al. DoRA, ICML 2024

GitHub: https://github.com/arnavtevatia/lora-dora-gpt2

1. Introduction

Fine-tuning large pretrained language models updates all parameters, which is memory-intensive and prone to overfitting on small datasets. For a 117M parameter model like GPT-2, the Adam optimizer alone requires ~936 MB of state. **LoRA** (Hu et al., ICLR 2022) addresses this by freezing pretrained weights and injecting trainable low-rank matrix pairs *BA* into transformer layers. The adapted forward pass is $h = W_0x + (\alpha/r)\cdot BAx$, where B is d×r and A is r×d with rank r much smaller than d. B initializes to zeros and A to Gaussian, ensuring the LoRA term is zero at initialization. Only A and B are trained.

DoRA (Liu et al., ICML 2024), my extension, decomposes pretrained weights into magnitude m and direction V and applies LoRA only to the directional component: $W' = m\cdot(W_0 + BA)/\|W_0 + BA\|_c$. This decouples magnitude and direction updates, enabling learning patterns closer to full fine-tuning while preserving LoRA's parameter efficiency and zero inference overhead.

2. Chosen Result

I replicate LoRA's core claim from Table 11 of the paper: LoRA achieves validation perplexity close to full fine-tuning on GPT-2 with significantly fewer trainable parameters. Specifically, at r=4 applied to W_q and W_v, LoRA trains ~0.12% of GPT-2's parameters while matching or exceeding full fine-tuning performance. This result is central because it demonstrates that parameter-efficient fine-tuning can match or exceed full fine-tuning performance under constrained data regimes. I also replicate the rank sensitivity finding from Table 18, which shows diminishing returns past r=4.

As my extension, I add DoRA at matching rank configurations (r=4, r=8) to test whether the magnitude-direction decomposition improves over LoRA at the same parameter budget. The DoRA paper (Table 1) reports consistent gains over LoRA across LLaMA-7B/13B; I test whether this holds on a smaller model with a simpler task.

3. Methodology

Model and dataset. GPT-2 small (117M parameters) loaded from HuggingFace. WikiText-2 (wikitext-2-raw-v1), tokenized into 512-token blocks with no padding, causal language modeling objective. Metric: validation perplexity = exp(avg cross-entropy loss per token).

Implementation. All adapters implemented from scratch in PyTorch without any PEFT library. Three core classes:

- **LoRALinear:** wraps a frozen nn.Linear, registers trainable A (r×d, Gaussian init) and B (d×r, zeros init), forward pass computes $W_0x + (\alpha/r)\cdot(xA^T)B^T$.
- **DoRALinear:** extends LoRALinear with a magnitude vector m initialized to column norms of W_0. Forward pass: adapted_weight = $W_0 + (\alpha/r)BA$; $W' = m\cdot(\text{adapted\_weight} / \|\text{adapted\_weight}\|_c)$. Column norms are detached from the gradient graph (DoRA Section 4.3), reducing training memory ~24% with <0.2% accuracy impact.
- **GPT2AttentionWithLoRA:** intercepts GPT-2's fused c_attn (Conv1D producing Q, K, V concatenated) and adds LoRA/DoRA deltas to the Q slice (first 768 values) and V slice (last 768 values) only, leaving K unchanged per LoRA Table 5.

Verification. Before training: (1) parameter audit confirming only A, B, and magnitude vectors have requires_grad=True; (2) numerical equivalence check verifying model output identical to pretrained GPT-2 at initialization (B=zeros ensures zero LoRA term).

Training. AdamW optimizer, gradient clipping at max_norm=1.0, 3 epochs. Learning rate 3e-4 for all runs, batch size 4, block size 512. Six configurations run sequentially on a single T4 GPU.

Table 1. Results across all six configurations.

Configuration	Trainable Params	Val PPL	vs Full FT	Peak GPU (MB)	Time/Epoch (s)
Frozen GPT-2	0	36.08	+3.95	N/A	N/A
Full Fine-tune	124,440K	32.13	--	5,753	728.6
LoRA r=4	147K	25.65	-6.48	4,118	532.8
LoRA r=8	295K	25.26	-6.87	4,120	533.5
DoRA r=4	**166K**	**25.50**	**-6.63**	4,227	578.7
DoRA r=8	**313K**	**25.22**	**-6.91**	4,228	580.8

4. Results and Analysis

LoRA vs full fine-tuning. One of the more surprising results I observed is that LoRA r=4 achieves a validation perplexity of 25.65, compared to 32.13 for full fine-tuning, while using 843x fewer trainable parameters. This does not appear to be due to failed convergence. As shown in Figure 2, full fine-tuning initially improves (reaching 27.76 after epoch 1), but then degrades to 32.13 by epoch 3, suggesting overfitting. A likely explanation is dataset size. WikiText-2 contains only ~2M tokens, which is relatively small compared to the 124M parameters being updated in full fine-tuning. LoRA's low-rank constraint restricts updates to a small subspace, which in this setting appears to act as implicit regularization. This is consistent with prior work showing that LoRA can outperform full fine-tuning in low-data regimes.

DoRA vs LoRA. Across all configurations I tested, DoRA performs slightly better than LoRA at the same rank: DoRA r=4 achieves 25.50 vs LoRA r=4 at 25.65, and DoRA r=8 achieves 25.22 vs LoRA r=8 at 25.26. The improvement is small but consistent across both ranks and all training epochs. This matches the intuition behind DoRA: decoupling magnitude and direction allows updates that more closely resemble those in full fine-tuning. That said, the improvement is noticeably smaller than what the DoRA paper reports on LLaMA-7B (+3.7 accuracy points). A likely reason is that GPT-2 on WikiText-2 is a relatively simple setting where the model is already close to its performance ceiling. In more complex tasks, this added flexibility would likely matter more.

Memory and efficiency. LoRA reduces peak GPU memory from 5,753 MB to 4,118 MB (about 28%), largely due to the smaller optimizer state. DoRA introduces only ~110 MB of overhead, since the additional magnitude parameters are minimal and the norm-detachment optimization keeps backprop costs low.

Rank sensitivity. Increasing rank from r=4 to r=8 improves performance for both LoRA and DoRA, but the gains are small (0.39 PPL for LoRA, 0.28 for DoRA). This reproduces the diminishing returns trend in LoRA Table 18 and supports the idea that only a low intrinsic rank is needed for this task.

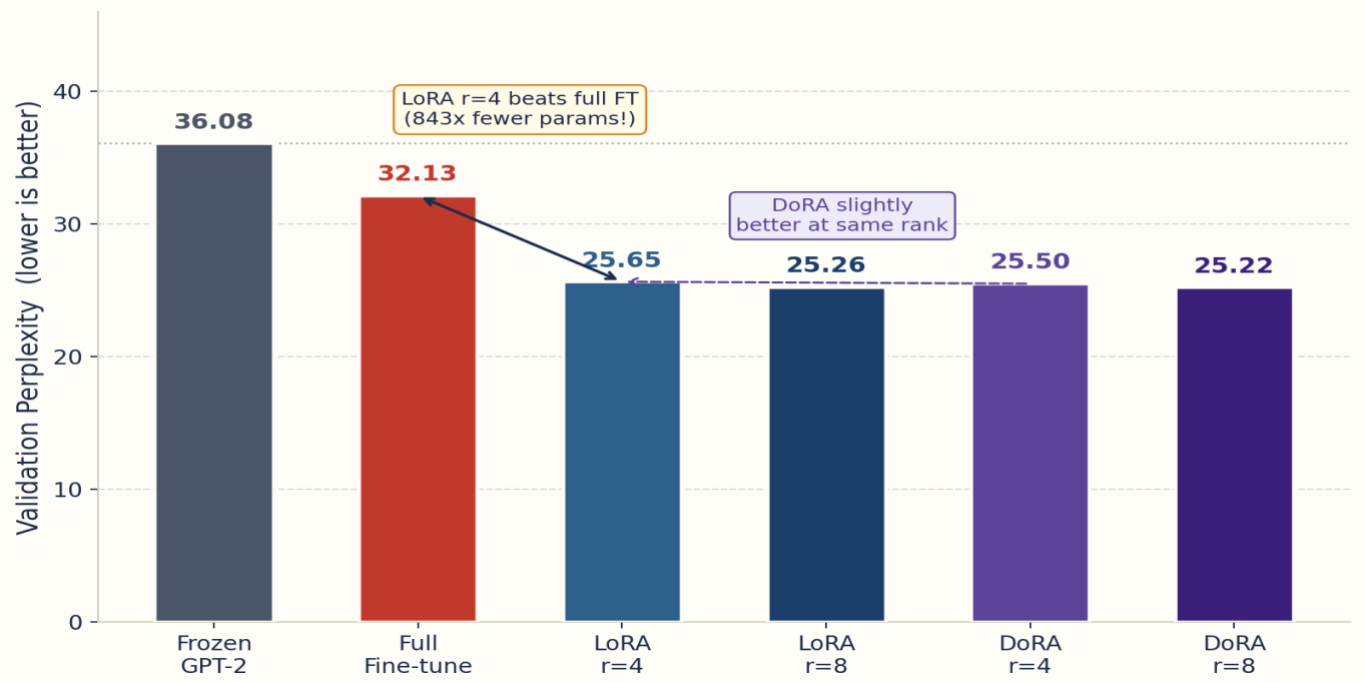

Fig 1. Validation perplexity by configuration. LoRA and DoRA beat full fine-tuning. All adapters outperform the frozen baseline.

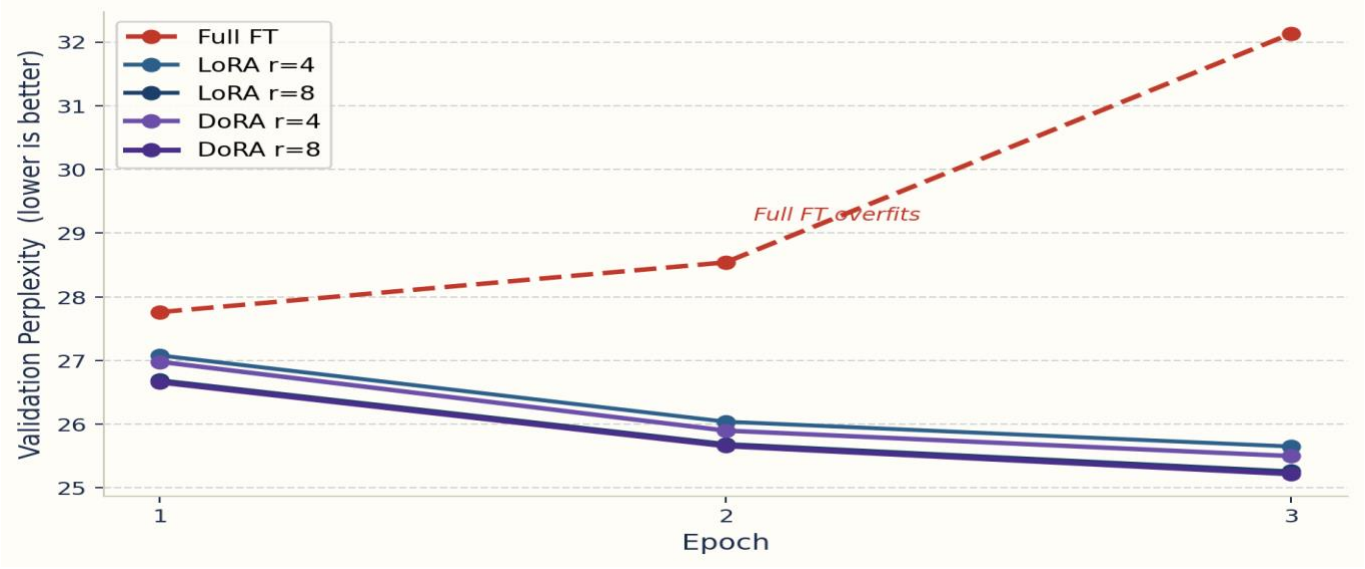

Fig 2. Training curves. Full FT overfits after epoch 1: validation perplexity rises from 27.76 to 32.13. LoRA and DoRA converge cleanly.

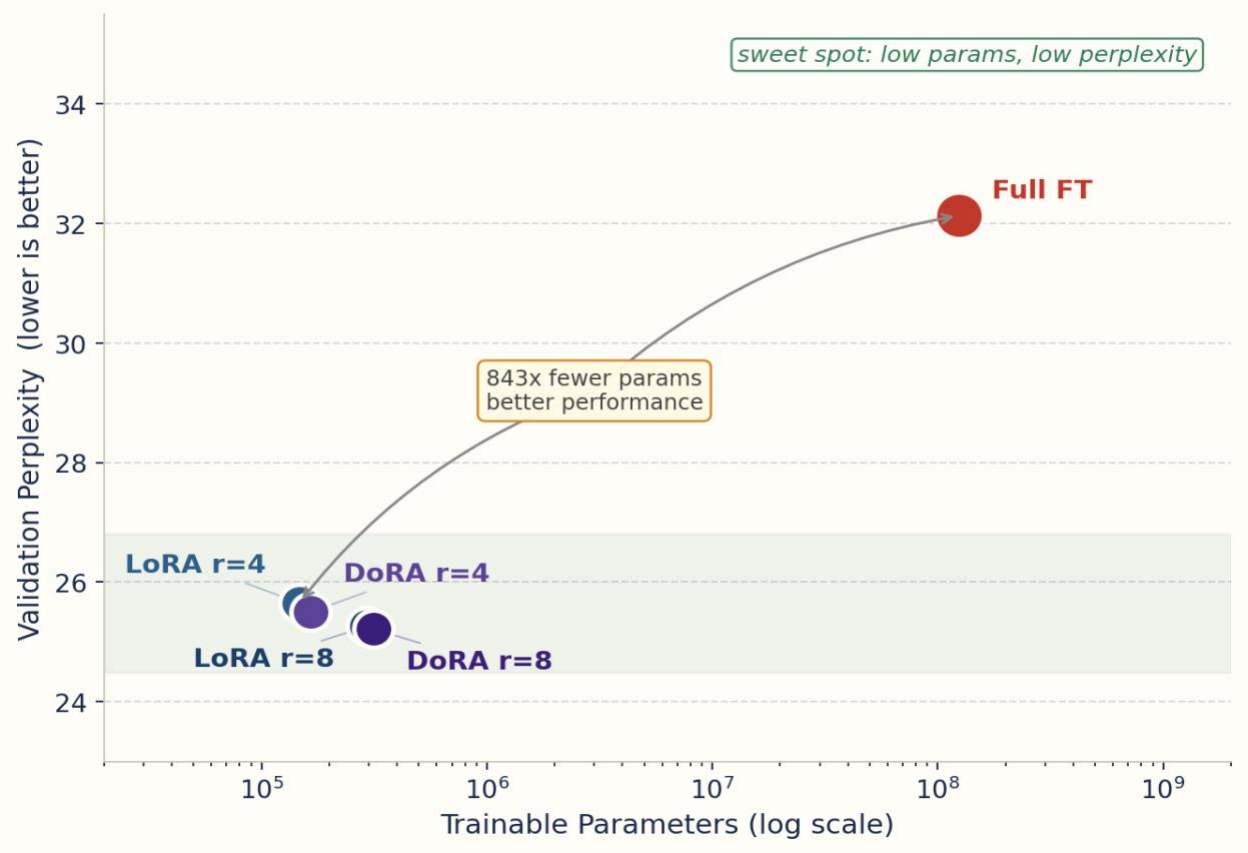

Fig 3. Parameter efficiency. LoRA and DoRA cluster bottom-left: low params, low perplexity.

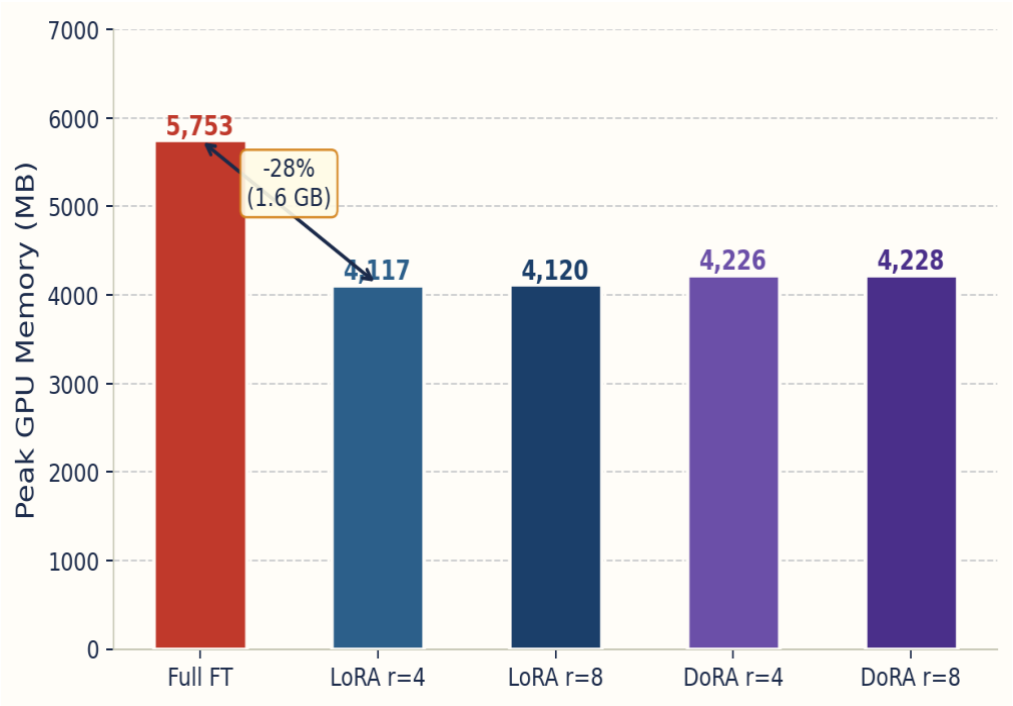

Fig 4. Peak GPU memory. LoRA saves 1.6 GB vs full FT. DoRA adds only ~110 MB over LoRA.

5. Reflections

Key lessons. The main takeaway from this project is how significant the implicit regularization effect of LoRA can be in practice. Going in, I expected LoRA to roughly match full fine-tuning, but instead it performed substantially better. This seems to be driven almost entirely by overfitting in full fine-tuning rather than any limitation of the method itself. It changes how I think about LoRA's value: it is not just about parameter efficiency, but also about more stable training in low-data settings.

Implementation challenges. The main engineering challenge was working with GPT-2's fused c_attn layer. Since Q, K, and V are generated through a single Conv1D projection, I could not directly replace individual components with LoRA modules. Instead, I computed the full QKV output from the frozen layer and applied LoRA/DoRA updates only to the Q and V slices. One detail that turned out to be important was verifying that the modified model matched the original GPT-2 output at initialization, which helped catch a subtle bug related to weight transposition early on.

DoRA observations. While DoRA consistently outperformed LoRA in my experiments, the gap was small. This is still informative: the larger improvements in the original paper likely depend on more complex tasks and larger models, where the ability to separately control magnitude and direction becomes more critical. In this setting, the added flexibility does not seem as necessary, but it still provides a small, consistent improvement at minimal extra cost. Based on these results, DoRA appears to be a reasonable default over LoRA in similar low-complexity settings, though I would want to test this claim more broadly before generalizing.

Limitations. A few caveats worth noting: my experiments use a single small dataset (WikiText-2, ~2M tokens), one model size (GPT-2 small), and only 3 training epochs. Results may not generalize to larger models, harder tasks, or longer training runs. The DoRA gap in particular warrants testing at scale before drawing strong conclusions.

Future directions. Three natural next steps: (1) apply LoRA and DoRA to FFN layers in addition to attention; (2) test the DoRA† (half-rank) variant to compare parameter efficiency more directly; (3) scale to GPT-2 medium or large to see whether the performance gap between LoRA and DoRA increases. These would help test whether the trends I observed here hold in more realistic, higher-capacity settings.

6. References

[1] Hu, E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022.* arXiv:2106.09685

[2] Liu, S., Wang, C., Yin, H., Molchanov, P., Wang, Y., Cheng, K., & Chen, M. (2024). DoRA: Weight-Decomposed Low-Rank Adaptation. *ICML 2024.* arXiv:2402.09353

[3] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. GPT-2. OpenAI.

[4] Merity, S., Xiong, C., Bradbury, J., & Socher, R. (2016). Pointer sentinel mixture models. WikiText-2. arXiv:1609.07843

[5] Wolf, T. et al. (2020). HuggingFace Transformers. *EMNLP 2020.* arXiv:1910.03771